

Documentación Técnica del Compilador C-TDS

Analizador Léxico, Sintáctico y Representación del AST

Taller de Diseño de Software (Cod. 3306) — TDS26

Septiembre de 2026

Abstract

Este documento describe el estado actual del compilador de C-TDS: el analizador léxico (`lexer.1`), el analizador sintáctico (`bison.y`) y la representación del Árbol de Sintaxis Abstracta (`ast.h` / `ast.c`). El objetivo no es solo describir *qué* hace cada archivo, sino *por qué* está hecho así: qué parte de la letra del proyecto o de la gramática del lenguaje motivó cada decisión, y qué problemas concretos aparecieron (y cómo se resolvieron) al implementarlo. La última sección está pensada específicamente para que el resto del equipo pueda continuar agregando reglas a `bison.y` siguiendo el mismo patrón que las que ya están resueltas.

Contents

1	Introducción	2
2	Analizador Léxico (<code>lexer.1</code>)	2
2.1	Palabras reservadas	2
2.2	Identificadores	2
2.3	Números: enteros y reales	2
2.4	Comentarios	3
2.5	Número de línea en los mensajes	3
2.6	Errores léxicos	3
3	Analizador Sintáctico (<code>bison.y</code>)	3
3.1	Estado actual: alcance deliberadamente reducido	3
3.2	Por qué <code>main</code> no es un token	4
3.3	Un problema real encontrado: ambigüedad <code>Type ID</code>	4
3.4	Precedencia de operadores	5
4	Representación del AST (<code>ast.h</code> y <code>ast.c</code>)	5
4.1	Origen de la estructura: <code>redesign_compiler.pdf</code>	5
4.2	Qué se agregó sobre ese documento, y por qué	5
4.3	<code>NodeList</code> : por qué existe y por qué no es parte del AST final	6
4.4	Las cuatro funciones, una por una	6
4.4.1	<code>newSymbol</code>	6
4.4.2	<code>newNode</code>	6
4.4.3	<code>newNodeList</code>	7
4.4.4	<code>attachChildren</code>	7
4.5	Por qué las cuatro funciones tienen el cuerpo vacío	7

1 Introducción

El compilador se construye de forma incremental: primero el análisis léxico y sintáctico, luego el semántico, y recién después la generación de código intermedio y objeto. Este documento cubre las dos primeras etapas, más la estructura de datos (el AST) que las conecta con las etapas siguientes.

El lenguaje fuente es C-TDS, especificado en el documento *Descripción del Lenguaje: C-TDS*. Todas las decisiones de diseño que no están fijadas explícitamente por ese documento se marcan como tales a lo largo de esta guía, junto con su justificación.

2 Analizador Léxico (`lexer.1`)

El analizador léxico convierte el texto fuente en una secuencia de *tokens*. Implementa exactamente el léxico definido en *Descripción del Lenguaje: C-TDS*, sección “Consideraciones del Léxico”.

2.1 Palabras reservadas

La letra define explícitamente el conjunto cerrado de palabras reservadas:

“Las palabras reservadas son: `boolean else false if int return true void while float`”

Cada una se reconoce con una regla propia que retorna el token correspondiente (`BOOLEAN`, `ELSE`, `IF`, etc.). Un punto importante: **main no es palabra reservada**. Aunque todo programa válido debe tener un método `main`, esa es una regla *semántica* (regla #3 de la sección “Reglas Semánticas”): *“Todo programa contiene la definición de un método llamado main”*. No es una regla léxica ni sintáctica, así que `main` matchea como un `ID` común, y su existencia se verificará más adelante contra la tabla de símbolos, en la etapa de análisis semántico.

2.2 Identificadores

La gramática del lenguaje define:

$$\langle id \rangle \rightarrow \langle alpha \rangle \langle alpha_num \rangle^* \quad \langle alpha_num \rangle \rightarrow \langle alpha \rangle \mid \langle digit \rangle$$

Es decir, un identificador es una letra seguida de cero o más letras/dígitos — **sin guión bajo**. La regla actual en `lexer.1` es:

```
1 ALPHA_NUM    { ALPHA } | { DIGIT } | "_"
```

Este `"_"` es una **extensión** sobre la gramática estricta. La letra permite explícitamente extensiones documentadas (*Descripción del Proyecto*, sección “Documentación”, punto 2: *“Una lista de decisiones de diseño, asumpciones, aclaraciones o extensiones realizadas en la etapa”*), así que esto es válido siempre que quede documentado como tal — que es lo que se hace acá. Si el equipo decide no permitir guiones bajos, basta con sacar el `"_"` de esa línea.

2.3 Números: enteros y reales

La gramática separa dos literales numéricos:

$$\langle int_literal \rangle \rightarrow \langle digit \rangle \langle digit \rangle^* \quad \langle float_literal \rangle \rightarrow \langle digit \rangle \langle digit \rangle^* . \langle digit \rangle \langle digit \rangle^*$$

Notar que *ambos* lados del punto exigen al menos un dígito. Esto se traduce directamente a dos definiciones separadas:

1	NUMBER	{DIGIT}+
2	FLOAT	{DIGIT}+"."{DIGIT}+

con la regla de **FLOAT** ubicada **antes** que la de **NUMBER** en el archivo. Esto importa: flex resuelve ambigüedades por longitud de coincidencia (*maximal munch*), pero ante un empate usa la regla que aparece primero en el archivo. Poner **FLOAT** primero evita cualquier duda sobre quién gana ese posible empate.

Como consecuencia directa de exigir dígitos a ambos lados del punto, entradas como **5.** o **.5** **no** son floats válidos: se tokenizan como **NUMBER** más un símbolo **.** inválido suelto. Esto se verificó explícitamente con casos de prueba y es el comportamiento esperado según la gramática, no un error.

2.4 Comentarios

La letra define dos formas de comentario:

“Dos tipos de comentarios son permitidos, aquellos que comienzan con `//` y terminan al final de la línea (únicamente pueden ser de una línea) y aquellos que están delimitados por `/*` y `*/` (pueden tener varias líneas de extensión).”

Ambos se descartan sin generar token, en línea con: “*Símbolos que no son tokens son descartados, por ejemplo, espacios, tabulaciones, nuevas líneas y comentarios*”.

2.5 Número de línea en los mensajes

Se activó `%option yylineno` y se agregó `[Linea %d]` a todos los mensajes de error. La letra no lo pide con esas palabras exactas, pero sí exige, en la sección “Consideraciones del Léxico”:

“Aquellos símbolos que no son permitidos en el lenguaje deben ser reportados, por ejemplo, `$`, `#`.”

Reportar un símbolo inválido sin decir en qué línea aparece es de poca utilidad práctica en un archivo real. Además, la siguiente etapa (el analizador sintáctico) necesita la línea para sus propios mensajes de error — `yyerror` en `bison.y` ya usa `yylineno`, que `lexer.l` mantiene actualizado. Es una decisión de diseño, documentada como tal.

2.6 Errores léxicos

La regla de error (`.` como última alternativa) reporta **ERROR LEXICO: simbolo no permitido** en vez del genérico **Token inválido** usado en versiones anteriores, para que el mensaje quede alineado con la terminología de la letra citada arriba.

3 Analizador Sintáctico (bison.y)

El analizador sintáctico verifica que la secuencia de tokens producida por el lexer respete la gramática de C-TDS, y construye el AST correspondiente a medida que reconoce cada producción.

3.1 Estado actual: alcance deliberadamente reducido

Este archivo no implementa toda la gramática del lenguaje. Implementa únicamente 4 producciones, elegidas para servir de patrón de referencia:

1. **Type** — nodo hoja, sin hijos.
2. **VarDecl** (con **IdList**) — nodo con un hijo fijo más una cantidad variable de hijos.
3. **Statement** (caso asignación) — nodo binario con **left/right**.
4. **Expr** (caso **NUMBER**) — nodo hoja con un valor literal.

El resto de la gramática (**Program** real, **MethodDecl**, **Block**, **if/while**, llamadas a método, y el resto de los operadores de **Expr**) **no está declarado todavía**.

El símbolo inicial actual es un **Program** de prueba mínimo (**VarDecl Statement**), útil únicamente para poder compilar y probar las 4 reglas ya escritas. **No es la gramática real del lenguaje** y se va a reemplazar cuando se implemente el **Program** verdadero (sección 3.3).

3.2 Por qué **main** no es un token

Se sacó deliberadamente un token **MAIN** que existía en una versión anterior del archivo. Como se explicó en la sección del lexer, la existencia de un método **main** es una regla semántica, no sintáctica: **main** debe llegar al parser como un ID normal, igual que cualquier otro identificador.

3.3 Un problema real encontrado: ambigüedad **Type ID**

Al diseñar cómo sería el **Program** real, apareció un problema concreto que vale la pena documentar en detalle, porque quien complete esta regla se va a topar con él si transcribe la gramática de forma literal.

La gramática de *Descripción del Lenguaje: C-TDS* define:

$$\langle \text{program} \rangle \rightarrow \langle \text{var_decl} \rangle^* \langle \text{method_decl} \rangle^*$$

Una traducción directa a Bison usaría dos no-terminales separados, uno para la lista de **var_decl** y otro para la lista de **method_decl**. El problema es que **ambas producciones empiezan igual**:

$$\langle \text{var_decl} \rangle \rightarrow \langle \text{type} \rangle \langle \text{id} \rangle^+ ; \quad \langle \text{method_decl} \rangle \rightarrow \langle \text{type} \rangle \langle \text{id} \rangle (\dots) \langle \text{block} \rangle$$

Un parser LALR(1) — como el que genera Bison — decide qué regla aplicar mirando únicamente *un* símbolo por delante (*lookahead*). Al ver **Type ID**, no hay forma de saber todavía si lo que sigue es **;**, **,** (entonces es una variable) o **(** (entonces es un método) sin seguir leyendo. Al declarar **VarDecl** y **MethodDecl** como listas separadas, Bison reporta un conflicto *shift/reduce* y lo resuelve siempre a favor de la variable — con el resultado de que **ningún método con tipo de retorno distinto de void llega a parsear jamás**. Este bug se detectó probando explícitamente contra el ejemplo de programa que trae la propia letra del lenguaje.

Solución. Unificar ambas producciones bajo un único no-terminal (por ejemplo **Declaration**, con **DeclList** como la lista que acumula ambos casos indistintamente) y decidir recién *después* de leer el identificador si continúa como declaración de variable o de método. Esto no cambia el lenguaje reconocido — sigue aceptando exactamente los mismos programas — pero elimina la ambigüedad estructural. Es la solución estándar para prefijos compartidos en gramáticas LL/LALR.

Quién implemente Program/MethodDecl/Block debe tener esto presente y no separar **VarDecl** de **MethodDecl** como dos listas independientes al nivel de programa. (Dentro de un **Block** sí es seguro usar solo **VarDecl**, porque la gramática no permite declarar métodos dentro de un bloque.)

3.4 Precedencia de operadores

La tabla de precedencia de `bison.y` reproduce exactamente la de *Descripción del Lenguaje: C-TDS*, sección “Expresiones” (de menor a mayor precedencia):

Operadores	Asociatividad
<code> </code>	izquierda
<code>&&</code>	izquierda
<code>==</code>	izquierda
<code>< ></code>	izquierda
<code>+ -</code>	izquierda
<code>* / %</code>	izquierda
<code>! (negación), - (unario)</code>	derecha

El `%prec UMINUS` se usa (cuando se implemente la regla de menos unario en `Expr`) para distinguir la resta binaria del menos unario, que comparten el mismo símbolo `-` pero tienen precedencias distintas.

4 Representación del AST (`ast.h` y `ast.c`)

4.1 Origen de la estructura: `redesign_compiler.pdf`

El diseño de `NodeAST` y `Symbol` sigue el documento de diseño del equipo:

```
1 typedef struct Symbol {
2     char *id;
3     char *value;
4 } Symbol;
5
6 typedef struct NodeAST {
7     Symbol *symbol;
8     struct NodeAST *left;
9     struct NodeAST *right;
10    DataType type;
11    NodeType nodeType;
12 } NodeAST;
```

Dos decisiones de ese documento vale la pena resaltar porque no son obvias a primera vista:

- **El tipo no vive en `Symbol`, vive en `NodeAST.type`.** Esto evita tener dos fuentes de verdad distintas para el tipo de una expresión — si estuviera en ambos lugares, podrían desincronizarse.
- **`Symbol` no es exclusivamente para variables.** También se usa para representar operadores o resultados temporales durante la resolución del AST (por ejemplo, un `BINOP_NODE` para `a + b` puede tener `symbol = {id: "+", value: "t1"}`, donde `t1` es un temporal). Esto es relevante para cuando se implemente el generador de código intermedio: los temporales de tres direcciones ya tienen un lugar natural donde vivir.

4.2 Qué se agregó sobre ese documento, y por qué

`redesign_compiler.pdf` ejemplificó la resolución de identificadores con un subconjunto reducido de casos (asignación, operación binaria, constante, retorno, definición). La gramática completa del lenguaje necesita más que eso. Concretamente, sobre `NodeAST` se agregó:

```

1 struct NodeAST **children;
2 int childCount;

```

porque varias producciones de la gramática tienen una cantidad **variable** de hijos, no fija en dos (*left/right* alcanza para nodos binarios, pero no para un programa con N declaraciones, un método con N parámetros, o una llamada con N argumentos). El campo `TYPE_FLOAT` en `DataType` también se agregó porque el documento original solo contemplaba `INT/BOOL/VOID`, mientras que la letra del lenguaje define explícitamente: “*Los tipos básicos en C-TDS son int, float y boolean*”.

Estado actual (versión mínima): el enum `NodeType` en `ast.h` solo declara los 5 valores que las 4 reglas ya implementadas en `bison.y` necesitan: `DEFINITION_NODE`, `VAR_DECL_NODE`, `ID_NODE`, `ASSIGNMENT_NODE` y `CONSTANT_NODE`. A medida que se implementen más reglas de la gramática, va a hacer falta agregar los `NodeType` correspondientes, por ejemplo:

- `METHOD_DECL_NODE`, `BLOCK_NODE`
- `IF_NODE`, `WHILE_NODE`
- `METHOD_CALL_NODE`, `UNARYOP_NODE`, `PARAM_NODE`

Se decide en el momento de implementar cada regla, no antes.

4.3 NodeList: por qué existe y por qué no es parte del AST final

Varias producciones de la gramática son listas (*cero o más, uno o más separados por coma*). Bison construye el resultado de una regla *de abajo hacia arriba*: cuando se reduce `IdList: IdList ', ' ID`, todavía no existe el nodo padre (`VarDecl`) que va a contener esa lista como hijos. Hace falta algún lugar transitorio donde acumular esos nodos hermanos mientras se van reconociendo, uno por uno, de izquierda a derecha.

`NodeList` cumple exactamente ese rol: es una lista enlazada simple, usada únicamente durante la construcción (dentro de las acciones de `bison.y`). Una vez que el nodo padre existe, `attachChildren` vuelca esa lista al arreglo `children[]` del padre y fija `childCount`. Ningún `NodeAST` del árbol final contiene un `NodeList`: es puramente un andamiaje de construcción, no una estructura de datos del AST en sí.

4.4 Las cuatro funciones, una por una

4.4.1 newSymbol

```

1 Symbol *newSymbol(const char *id, const char *value);

```

Crea un `Symbol` reservado dinámicamente. El detalle de implementación más importante: **debe copiar** las cadenas recibidas (típicamente con `strdup`), no quedarse con el puntero original. La razón es que `id` normalmente viene de `yytext` (el lexema que reconoció flex), y flex **reutiliza el mismo buffer** en cada token siguiente. Si `Symbol` guardara el puntero crudo en vez de una copia, el contenido cambiaría por debajo sin aviso apenas el lexer procesa el próximo token, corrompiendo silenciosamente todos los `Symbol` creados hasta ese momento.

4.4.2 newNode

```

1 NodeAST *newNode(NodeType nodeType, Symbol *symbol,
2                 NodeAST *left, NodeAST *right);

```

Crea un `NodeAST`, dejando `children == NULL` y `childCount == 0` (se completan aparte, ver `attachChildren`, solo en los nodos que los necesitan). El campo `type` se espera en un valor por defecto: el tipo *real* de la mayoría de los nodos no se conoce en el momento sintáctico — recién se resuelve en el análisis semántico, cuando se puede consultar la tabla de símbolos para saber,

por ejemplo, con qué tipo fue declarada una variable. Los pocos casos donde el tipo *sí* se conoce de entrada (una constante entera es siempre `TYPE_INT`, un `Type: int` siempre da `TYPE_INT`) lo fijan explícitamente después de llamar a `newNode`, como se ve en las reglas `Type` y `Expr: NUMBER` de `bison.y`.

4.4.3 `newNodeList`

```
1 NodeList *newNodeList(NodeAST *node, NodeList *next);
```

Crea (o extiende) una celda de la lista transitoria descripta arriba. El patrón de uso en `bison.y`, visible en la regla `IdList`, es: crear una lista de un elemento en el primer caso base, y en el caso recursivo recorrer hasta el final de la lista existente para agregar el nuevo elemento, preservando el orden textual en que aparecen en el código fuente.

4.4.4 `attachChildren`

```
1 void attachChildren(NodeAST *parent, NodeList *list);
```

Centraliza el patrón “contar los elementos de la lista, reservar el arreglo del tamaño exacto, copiar cada nodo en orden” que, sin esta función, se repetiría casi idéntico en cada regla de la gramática que tenga una cantidad variable de hijos. Se usó en `VarDecl` y se va a volver a usar en `MethodDecl` (para los parámetros), `MethodCall` (para los argumentos) y `Program` (para la lista de declaraciones), entre otras.

4.5 Por qué las cuatro funciones tienen el cuerpo vacío

Esto es deliberado, no un olvido. Se decidió definir primero la **interfaz completa** (qué recibe cada función, qué devuelve, quién es dueño de la memoria en cada caso — todo documentado en los comentarios de `ast.h`) y recién después completar la implementación real en `ast.c`. Esto permite que `bison.y` ya compile contra una interfaz estable mientras la implementación se sigue discutiendo o repartiendo entre el equipo, sin bloquear el trabajo de nadie.